Assignment-3

UCS420 – "Cognitive Computing"
Pandas – Solved

Q1. Create a dataset as follows in the table.

```
import pandas as pd

data = {
    "Tid": [1,2,3,4,5,6,7,8,9,10],
    "Refund": ["Yes","No","No","Yes","No","No","Yes","No","No","No"],
    "Marital_Status": ["Single","Married","Single","Married","Divorced",
                       "Married","Divorced","Single","Married","Single"],
    "Taxable_Income": ["125K","100K","70K","120K","95K",
                       "60K","220K","85K","75K","90K"],
    "Cheat": ["No","No","No","No","Yes","No","No","Yes","No","Yes"]
}
df1 = pd.DataFrame(data)
print(df1)
```

Output:

```
   Tid Refund Marital_Status Taxable_Income Cheat
0    1    Yes         Single           125K    No
1    2     No        Married           100K    No
2    3     No         Single            70K    No
3    4    Yes        Married           120K    No
4    5     No       Divorced            95K   Yes
5    6     No        Married            60K    No
6    7    Yes       Divorced           220K    No
7    8     No         Single            85K   Yes
8    9     No        Married            75K    No
9   10     No         Single            90K   Yes
```

Q2. Locate row 0, 4, 7 and 8 using DataFrame.

```
result = df1.loc[[0, 4, 7, 8]]
print(result)
```

Output:

```
   Tid Refund Marital_Status Taxable_Income Cheat
0    1    Yes         Single           125K    No
4    5     No       Divorced            95K   Yes
7    8     No         Single            85K   Yes
8    9     No        Married            75K    No
```

Q3. Navigate the DataFrame.

1. Select row from index 3 to 7.

```
print(df1.loc[3:7])
   Tid Refund Marital_Status Taxable_Income Cheat
3    4    Yes        Married           120K    No
4    5     No       Divorced            95K   Yes
5    6     No        Married            60K    No
6    7    Yes       Divorced           220K    No
7    8     No         Single            85K   Yes
```

2. Select row from index 4 to 8, and column 2 to 4.

```
print(df1.iloc[4:9, 2:5])
  Marital_Status Taxable_Income Cheat
4       Divorced            95K   Yes
5        Married            60K    No
6       Divorced           220K    No
7         Single            85K   Yes
8        Married            75K    No
```

3. Select all rows with column index 1 to 3 (include index 3).

```
print(df1.iloc[:, 1:4])
```

	Refund	Marital_Status	Taxable_Income
0	Yes	Single	125K
1	No	Married	100K
2	No	Single	70K
3	Yes	Married	120K
4	No	Divorced	95K
5	No	Married	60K
6	Yes	Divorced	220K
7	No	Single	85K
8	No	Married	75K
9	No	Single	90K

Q4. Read a csv file and display its first five rows. (Iris dataset)

```
import pandas as pd

# iris.csv downloaded from Kaggle (uciml/iris) and placed in working directory
iris = pd.read_csv("iris.csv")
print(iris.head())
```

Output:

```
   Id  sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  species
0   1                5.1               3.5                1.4               0.2   setosa
1   2                4.9               3.0                1.4               0.2   setosa
2   3                4.7               3.2                1.3               0.2   setosa
3   4                4.6               3.1                1.5               0.2   setosa
4   5                5.0               3.6                1.4               0.2   setosa
```

Q5. From the csv file (Q4), delete row 4 and delete column 3. Display the result.

```
iris_mod = iris.drop(index=4)                       # delete row with index 4
iris_mod = iris_mod.drop(iris_mod.columns[3], axis=1)  # delete column at position 3
print(iris_mod.head())
```

Output:

```
   Id  sepal length (cm)  sepal width (cm)  petal width (cm)  species
0   1                5.1               3.5               0.2   setosa
1   2                4.9               3.0               0.2   setosa
2   3                4.7               3.2               0.2   setosa
3   4                4.6               3.1               0.2   setosa
5   6                5.4               3.9               0.4   setosa
```

Q6. Create a sample dataset (employees.csv).

```
import pandas as pd

emp_data = {
    "Employee_ID": [101, 102, 103, 104, 105],
    "Name": ["Alice","Bob","Charlie","Diana","Edward"],
    "Department": ["HR","IT","IT","Marketing","Sales"],
    "Age": [29, 34, 41, 28, 38],
    "Salary": [50000, 70000, 65000, 55000, 60000],
    "Years_of_Experience": [4, 8, 10, 3, 12],
    "Joining_Date": ["2020-03-15","2017-07-19","2013-06-01","2021-02-10","2010-11-25"],
    "Gender": ["Female","Male","Male","Female","Male"],
    "Bonus": [5000, 7000, 6000, 4500, 5000],
    "Rating": [4.5, 4.0, 3.8, 4.7, 3.5]
}
emp = pd.DataFrame(emp_data)
emp.to_csv("employees.csv", index=False)
print(emp)
```

	Employee_ID	Name	Department	Age	Salary	Years_of_Experience	Joining_Date	Gender	Bonus	Rating
0	101	Alice	HR	29	50000	4	2020-03-15	Female	5000	4.5
1	102	Bob	IT	34	70000	8	2017-07-19	Male	7000	4.0
2	103	Charlie	IT	41	65000	10	2013-06-01	Male	6000	3.8
3	104	Diana	Marketing	28	55000	3	2021-02-10	Female	4500	4.7
4	105	Edward	Sales	38	60000	12	2010-11-25	Male	5000	3.5

a) Shape of the DataFrame.

```
print(emp.shape)
(5, 10)
```

b) Summary - data types and non-null counts.

```
print(emp.info())
<class 'pandas.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 10 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   Employee_ID          5 non-null      int64
 1   Name                 5 non-null      object
 2   Department           5 non-null      object
 3   Age                  5 non-null      int64
 4   Salary               5 non-null      int64
 5   Years_of_Experience  5 non-null      int64
 6   Joining_Date         5 non-null      object
 7   Gender               5 non-null      object
 8   Bonus                5 non-null      int64
 9   Rating               5 non-null      float64
dtypes: float64(1), int64(5), object(4)
memory usage: 532.0 bytes
```

c) Generate descriptive statistics.

```
print(emp.describe())
       Employee_ID    Age    Salary  Years_of_Experience   Bonus   Rating
count         5.00   5.00      5.00                 5.00     5.0     5.00
mean        103.00  34.00  60000.00                 7.40  5500.0     4.10
std           1.58   5.61   7905.69                 3.85  1000.0     0.49
min         101.00  28.00  50000.00                 3.00  4500.0     3.50
25%         102.00  29.00  55000.00                 4.00  5000.0     3.80
50%         103.00  34.00  60000.00                 8.00  5000.0     4.00
75%         104.00  38.00  65000.00                10.00  6000.0     4.50
max         105.00  41.00  70000.00                12.00  7000.0     4.70
```

d) First 5 rows and last 3 rows.

```
print(emp.head())
print(emp.tail(3))
```

head() -> same as full table shown above. tail(3) output:

	Employee_ID	Name	Department	Age	Salary	Years_of_Experience	Joining_Date	Gender	Bonus	Rating
2	103	Charlie	IT	41	65000	10	2013-06-01	Male	6000	3.8
3	104	Diana	Marketing	28	55000	3	2021-02-10	Female	4500	4.7
4	105	Edward	Sales	38	60000	12	2010-11-25	Male	5000	3.5

e) Statistics.

```
avg_salary   = emp["Salary"].mean()
total_bonus  = emp["Bonus"].sum()
youngest_age = emp["Age"].min()
highest_rate = emp["Rating"].max()
print(avg_salary, total_bonus, youngest_age, highest_rate)
i.   Average Salary        = 60000.0
ii.  Total Bonus Paid      = 27500
iii. Youngest Employee Age  = 28
iv.  Highest Rating        = 4.7
```

f) Sort by Salary (descending).

```
emp_sorted = emp.sort_values(by="Salary", ascending=False)
print(emp_sorted)
   Employee_ID     Name Department  Age  Salary  Years_of_Experience Joining_Date  Gender  Bonus  Rating
1          102      Bob         IT   34   70000                    8   2017-07-19    Male   7000     4.0
2          103  Charlie         IT   41   65000                   10   2013-06-01    Male   6000     3.8
4          105   Edward      Sales   38   60000                   12   2010-11-25    Male   5000     3.5
3          104    Diana  Marketing   28   55000                    3   2021-02-10  Female   4500     4.7
0          101    Alice         HR   29   50000                    4   2020-03-15  Female   5000     4.5
```

g) Categorize employees by performance rating.

```
def categorize(r):
    if r >= 4.5:
        return "Excellent"
    elif r >= 4.0:
        return "Good"
    else:
        return "Average"

emp["Performance_Category"] = emp["Rating"].apply(categorize)
print(emp[["Name","Rating","Performance_Category"]])
      Name  Rating Performance_Category
0    Alice     4.5            Excellent
1      Bob     4.0                 Good
2  Charlie     3.8              Average
3    Diana     4.7            Excellent
4   Edward     3.5              Average
```

h) Identify missing values.

```
print(emp.isnull().sum())
Employee_ID             0
Name                    0
Department              0
Age                     0
Salary                  0
Years_of_Experience     0
Joining_Date            0
Gender                  0
Bonus                   0
Rating                  0
Performance_Category    0
dtype: int64      (No missing values found)
```

i) Rename Employee_ID to ID.

```
emp = emp.rename(columns={"Employee_ID": "ID"})
print(emp.columns.tolist())
['ID', 'Name', 'Department', 'Age', 'Salary', 'Years_of_Experience',
 'Joining_Date', 'Gender', 'Bonus', 'Rating', 'Performance_Category']
```

j) i. Employees with more than 5 years experience.

```
print(emp[emp["Years_of_Experience"] > 5])
    ID     Name Department  Age  Salary  Years_of_Experience  ... Rating Performance_Category
1  102      Bob         IT   34   70000                    8  ...    4.0                 Good
2  103  Charlie         IT   41   65000                   10  ...    3.8              Average
4  105   Edward      Sales   38   60000                   12  ...    3.5              Average
```

j) ii. Employees belonging to the IT department.

```
print(emp[emp["Department"] == "IT"])
    ID     Name Department  Age  Salary  Years_of_Experience  ... Rating Performance_Category
1  102      Bob         IT   34   70000                    8  ...    4.0                 Good
2  103  Charlie         IT   41   65000                   10  ...    3.8              Average
```

k) Add Tax column (10% of Salary).

```
emp["Tax"] = emp["Salary"] * 0.10
print(emp[["Name","Salary","Tax"]])
```

```
      Name  Salary     Tax
0    Alice   50000  5000.0
1      Bob   70000  7000.0
2  Charlie   65000  6500.0
3    Diana   55000  5500.0
4   Edward   60000  6000.0
```

l) Save the modified DataFrame to a new CSV file.

```
emp.to_csv("employees_modified.csv", index=False)
print("Saved successfully as employees_modified.csv")
```

```
Saved successfully as employees_modified.csv
```